作品编号：CACR2026EA16XZ

作品类别： □软件设计 □硬件制作 ■工程实践

□密码应用技术 □其它

2026 年第十一届全国密码技术竞赛作品设计报告

作品题目：ML-KEM 解封装实现的时间侧信道检测与常数时间防护——多机制泄漏的预注册检测、交叉定位与防护复测

2026 年 10 月 21 日

中国密码学会

基本信息表

编号：CACR2026EA16XZ

作品题目：ML-KEM 解封装实现的时间侧信道检测与常数时间防护——多机制泄漏的预注册检测、交叉定位与防护复测

作品类别：□软件设计 □硬件制作 ■工程实践 □密码技术应用 □其它

作品内容摘要：

ML-KEM 解封装在 FO 变换下须对任意密文执行解密、重加密与比较，任何依赖秘密的分支、访存或可变时延指令都可能经由执行时间泄露信息。针对泄漏机制多样、效应仅数十纳秒、单一工具易误判的问题，本作品构建了面向 ML-KEM-768 便携 C 实现的“检测—定位—防护—复测”平台：以完整解封装计时窗口、可审计的锁频绑核控制和采集前冻结的检验计划为测量基础，以 Welch 检验、Holm 多重校正、跨运行合并与最小可检测效应（MDE）为统计判据，并联合反汇编、秘密标记 Memcheck、可变时延指令检查、Microwalk 与 Binsec 实现函数级定位。在两个真实历史缺陷（liboqs 编译器引入分支、KyberSlash 秘密除法）、两个人工校准夹具和一个阴性对照上的实验表明：liboqs 未修版本完整解封装均值差为 +32.3 ns（95% 置信区间 [20.3, 44.3]，约占 57 μs 的 0.06%），官方修复后区间含 0，开销 +1.51%；KyberSlash 除法可被定位层稳定识别，但在本机计时层未检出（MDE 25–67 ns）；阴性对照上运行的 6 种工具误报均为 0。90 格全流程排查还发现，KyberSlash 官方修复版本中仍保留与 clangover 同源的秘密相关分支，表明修复已知泄漏不等于产物常数时间。上述结论限定于 x86-64 单平台与本地白盒威胁模型。

关键词（五个）：ML-KEM；时间侧信道；常数时间实现；泄漏检测与定位；KyberSlash

目 录

1 作品功能与性能说明 4
1.1 研究背景与问题动机 4
1.2 威胁模型与评估边界 5
1.3 作品功能概述 6
1.4 主要创新点 6
1.5 主要结果一览 8
2 设计与实现方案 9
2.1 实现原理 9
2.2 平台总体架构 10
2.3 测量层设计 11
2.4 统计层设计 12
2.5 定位层设计 13
2.6 防护设计与泄露渠道审计 14
2.7 运行结果 16
2.8 技术指标 16
3 系统测试与结果 17
3.1 测试方案 17
3.2 功能测试 17
3.3 检测与定位结果 18
3.4 防护效果复测 21
3.5 功效、稳健性与异常输入对照 22
3.6 性能测试 23
3.7 跨编译选项与跨环境验证 24
3.8 多工具交叉矩阵 27
3.9 可复现性 28
4 与现有工作对比及应用前景 29
4.1 与现有工作的对比 29
4.2 应用前景 30
5 结论 31
5.1 主要结论 31
5.2 局限性 31
5.3 后续工作 32
参考文献 33
附录 A 题面要求对照 36
附录 B 证据索引 37

1 作品功能与性能说明

1.1 研究背景与问题动机

ML-KEM（FIPS 203）[1] 由 CRYSTALS-Kyber [2] 标准化而来，是当前后量子密码迁移中的核心密钥封装机制。为获得 IND-CCA2 安全性，ML-KEM 采用 Fujisaki-Okamoto（FO）变换：解封装时先用私钥解密得到消息 m′，再以 m′ 重新加密得到 c′，并将 c′ 与输入密文 c 比较；相等则输出共享密钥，不等则以隐式拒绝输出 J(z‖c)。这一结构决定了解封装必须对攻击者任意构造的密文执行“解密—重加密—比较”全过程，其中任何一处依赖秘密的条件分支、访存地址或可变时延指令，都可能把私钥或解密消息的信息投射到执行时间上。已有研究表明，FO 结构中重加密与比较环节的计时差异足以构成选择密文密钥恢复攻击 [10]；KyberSlash [3] 进一步证明，参考实现中以秘密值为被除数的除以 q 运算在多款处理器上导致可利用的时间差异，类似的可变时延除法问题也出现在 HQC 等方案中 [12]。

本作品关注的核心困难并不在于“是否存在泄漏机制”，而在于如何在真实编译产物上以可审计的方式回答以下四个问题：

（1）效应极小，测量噪声远大于信号。以本作品的 M1 为例，完整解封装均值约 57 μs，而泄漏导致的类间均值差约 32 ns，仅占约 0.06%。动态频率调节本身即可成为侧信道 [11]，若不控制频率、核迁移与预热，类间差异会被淹没或被漂移伪造。

（2）源码级常数时间不等于产物级常数时间。clangover [4][5] 显示，源码已写成算术掩码形式的 poly_frommsg，在特定 Clang 版本与优化选项下被编译器还原为按消息位跳转的指令；因此检测必须落到实际被计时的二进制上。

（3）单一工具结论不可靠。统计类工具（dudect [6]、TVLA [7][8]、RTLF [9]）只回答“两类时间分布是否可区分”，无法定位原因，且在漂移或类构造偏差下可能给出方向相反的结论；动态与形式化工具（ctgrind [16]、Microwalk [14]、Binsec/Rel [15]、ct-verif [13]）能定位或有界证明，但不回答“该机制在本机上是否可测”。开发者对这些工具的实际采用率也很低 [18]。

（4）“未检出”常被误读为“常数时间”。若不给出样本功效，阴性结论既无法与阳性结论对比，也无法指导防护优先级。

据此，本作品的目标是：针对 ML-KEM 解封装，建立一个能够把“检测—定位—防护—复测”串成闭环的平台，使每一个阳性结论都能落到具体函数和指令，每一个阴性结论都附带最小可检测效应（MDE），每一次防护都在同一规程下复测，并且整个过程在采集前预注册、可被第三方复算。

1.2 威胁模型与评估边界

本作品采用本地白盒评估模型（对应题面第 10 条），各要素见表 1。

表 1 威胁模型与评估边界

要素	本作品的设定
评估者能力	拥有被测实现源码，可自行编译并在同一台机器上运行；可选择公开测试输入（合法密文、篡改密文、截断或格式破损密文）；可重复调用解封装并读取每次调用的耗时
秘密对象	解封装私钥 dk 中参与解密的部分（ML-KEM-768 为 1152 B）、隐式拒绝秘密 z（32 B）、解密消息 m′ 及其派生量、重加密比较结果
可观测量	一次完整解封装调用的墙钟时间（CLOCK_MONOTONIC）；M4 夹具为单次比较调用时间；程序标准输出、标准错误与退出码（由泄露渠道审计单独检查）
不观测的量	共享密钥、私钥与内部验证标志均不输出；不使用功耗、电磁、故障注入等物理信道；不假设攻击者与被测进程共享缓存
攻击目标	判断时间分布是否随秘密相关的输入类别变化（可区分性），并定位导致差异的代码；不进行、也不交付密钥恢复
范围之外	远程网络场景；ARM 等其他处理器架构；共驻缓存攻击的专门实验；最终标准版之外的其他 ML-KEM 参数集

需要强调两点。第一，“可区分”不等于“可利用”：在本地锁频条件下测得约 32 ns 的差异，并不能推出远程网络环境下仍可观测，也不能推出可恢复密钥；本作品没有远程实验数据，因此不作此类推断。第二，M1 的对象是 FIPS 203 初始公开草案（ipd）对应实现，M2 是历史 Kyber768，二者均不等同于最终标准 ML-KEM-768；只有 M5 的 mlkem-native 对应最终标准。

1.3 作品功能概述

平台由五个被测模块和四个共享层组成（总体架构见 2.2 节图 2），提供以下功能：

（1）统一输入生成与类别构造。各模块共用确定性种子派生的输入，主类对为 message-zero 与 message-ones：两类均为合法、等长（1088 B）密文，区别仅在于解密后消息的每一位全为 0 或全为 1，从而把类间差异限定在秘密消息上；另设篡改密文、随机密文、尾部格式破损与截断等族外对照类。

（2）受控测量。统一 measure 入口只计时一次完整解封装调用；输入生成、内存分配、日志与 I/O 均在计时窗口外；类别乱序交替，窗口外预热 100 次，逐次原始值全量落盘，不删除普通长尾。锁频控制器负责关闭睿频、固定频率、绑核，并在放行前和退出后自动核对。

（3）统计检测。Welch 均值差检验与 95% 正态近似区间、预注册 Holm 家族、跨运行逆方差固定效应合并与 I^2 异质性、MDE 功效分析，以及 RTLF、online dudect、Mona 三种族外交叉检验。

（4）函数级定位。反汇编检查、秘密标记 Memcheck（ctgrind 思路）、KyberSlash 发布的 Valgrind 可变时延指令补丁（varlat）、Microwalk 轨迹互信息排序、Binsec 二进制有界常数时间检查，并对 3 个实现 ×5 个解封装步骤 ×6 类泄漏来源共 90 格逐格排查。

（5）防护与审计。采用并核验上游修复，同时提供常数时间比较、掩码选择等防护对照；对日志、异常与返回码做泄露渠道专项审计；在同一规程下复测，并测量性能开销与功能等价性。

（6）可复现。一键入口分 quick 与 full 两档，冻结计划、产物摘要、环境记录和证据索引随包提供。

1.4 主要创新点

结合题面要求与已有工作，本作品的贡献可归纳为以下四点。需要说明的是，所研究的两个真实缺陷均为已公开的历史问题，M3、M4 为人工构造的校准夹具，本作品不主张发现新漏洞。

（1）分层证据模型：把“机制存在”“热点可测”“端到端可测”区分开。本作品将每个模块的结论分别登记在定位层、热点微基准层和完整解封装层，并用 MDE 给出“未检出”的量化含义。KyberSlash 案例最能说明其必要性：三处秘密除法在定位层被稳定识别（varlat 1280 errors / 3 contexts，修复后 0/0），但在本机除法热点微基准层（合并 Δ = +0.044 ns，95% CI [−0.430, 0.517]）和完整解封装层（+4.58 ns，[−3.08, 12.25]）均未检出。若只看计时，会错误地认为“无问题”；若只看定位，会错误地认为“本机可测”。分层登记避免了这两种误读。

（2）预注册与可审计的锁频协议。检验计划（主类对、样本预算、停止条件、Holm 家族成员）在采集前以一次提交冻结，控制器在每次运行时校验计划摘要；锁频控制器在放行前做 20 次频率探针，退出后核对设置恢复。收尾阶段 117 次控制器运行全部通过放行与恢复核对。对照实验给出一个有实际意义的结果：在本机上锁频并没有显著降低变异系数（锁频/不锁频 CV 比为 0.964、1.006、1.089），M1 阳性与 M5 阴性的判定在两种条件下也一致。锁频的主要价值在于使测量条件确定、可核对、可复算，而不是“降噪”。这一结论修正了“锁频即可提高灵敏度”的常见直觉。

（3）多工具交叉定位与全流程排查，揭示“修复已知泄漏不等于产物常数时间”。在 9 种工具 ×5 个模块的矩阵中，30 格有真实运行产物；在指定阴性对照 M5 上运行过的 6 种工具误报均为 0；同时矩阵也暴露出单一工具的误读风险，例如 dudect 在 KyberSlash 已修复版本上报警，而未修复版本反而不报警。90 格排查进一步发现：KyberSlash 官方修复提交 272125f 的产物中，poly_frommsg 仍保留一条以消息位为条件的 bt/jae 分支，未修与已修两侧指令字节完全相同，秘密标记 Memcheck 在两侧各报 256 errors / 1 context。该分支与 clangover 同源。这是历史提交上的机制层观察，本作品未做逐指令时序归因，因此只将其作为 RTLF 在 M2 两臂均报警的候选解释，不作为新漏洞主张。

（4）编译优化级别与跨环境对照。对 M1 两版和 M5 在 -O0/-O2/-O3/-Os 四个级别共 12 格分别做产物判定和锁频计时：clangover 的 bt/jae 分支只在 -Os 出现，且只有 -Os 格的合并区间不含 0；-O2/-O3 虽存在另一种形态的秘密相关分支（Memcheck 命中 8 / 15 个位置），但计时层未检出；官方修复在

四个级别上均无秘密相关分支。另在三台不锁频的 WSL2 环境中复现，确定性证据（反汇编、Memcheck）与裸金属一致，而 M1 的约 30 ns 级效应在虚拟化噪声下不可检出。这为“产物级检测必须绑定编译条件”提供了直接证据。

1.5 主要结果一览

表 2 汇总五个模块在各证据层上的结论。所有数值来自本机（AMD Ryzen 9 9950X，Ubuntu 24.04.4，Linux 6.17，clang 18.1.3）锁频采集，每类 10000 次，三次独立运行合并。

表 2 五个模块的分层结论汇总

模块	对象与角色	定位层	完整解封装计时层（合并 Δ, 95% CI）	防护后
M1	liboqs 0.10.0 ML-KEM-768-ipd，真实历史案例（clangover）	阳性：poly_frommsg 中 bt/jae 分支；Memcheck 1 个秘密条件跳转	阳性：+32.311 ns [20.282, 44.339]；Holm p = 1.30×10^{-5}	官方修复 982c762：分支消失，−8.150 ns [−21.916, 5.616]，未检出
M2	pq-crystals Kyber768，真实历史案例（KyberSlash）	阳性：三处热点各 1 条 div；varlat 1280 / 3	未检出：+4.58 ns [−3.08, 12.25]；热点微基准 +0.044 ns [−0.430, 0.517]	272125f：div 计数 0，varlat 0/0；poly_frommsg 分支仍在
M3	秘密下标查表，人工校准	阳性：Memcheck 1280 / 2；Microwalk 互信息 1.922 bits	未检出：−0.73 ns [−5.56, 4.10]	off 臂：Memcheck 0/0，0.722 bits
M4	32 B 提前退出比较，人工校准	阳性：Memcheck 32 / 1；Binsec insecure	阳性：vulnerable 臂 −4.089 ns [−4.215, −3.963]	ct 臂：+0.055 ns [−0.363, 0.473]；Binsec 有界 secure
M5	mlkem-native b3ba7b3，阴性对照	未命中：三种模式各 10 次 0/0	未检出：−5.30 ns [−11.77, 1.16]	—（无需修复）

注：M4 为单次比较调用计时（CLOCK_MONOTONIC_RAW），与其余模块的完整解封装计时不可直接比较；M4 的家族窗口混合了泄漏臂与不泄漏臂（$I^2 = 98.6\%$），故只报告分臂数值。“未检出”指观测效应小于该预算下的 MDE，不等于常数时间证明。

2 设计与实现方案

2.1 实现原理

2.1.1 解封装流程与泄漏面

图 1 给出 ML-KEM 解封装的四个计算步骤以及本作品各模块的泄漏点位置。KyberSlash 的第一处除法位于解密步骤 poly_tomsg，第二、三处位于重加密步骤的 poly_compress 与 polyvec_compress；clangover 引入的分支和 M3 人工查表均位于重加密步骤的 poly_frommsg；M4 的提前退出比较对应比较步骤，作为独立夹具实现。

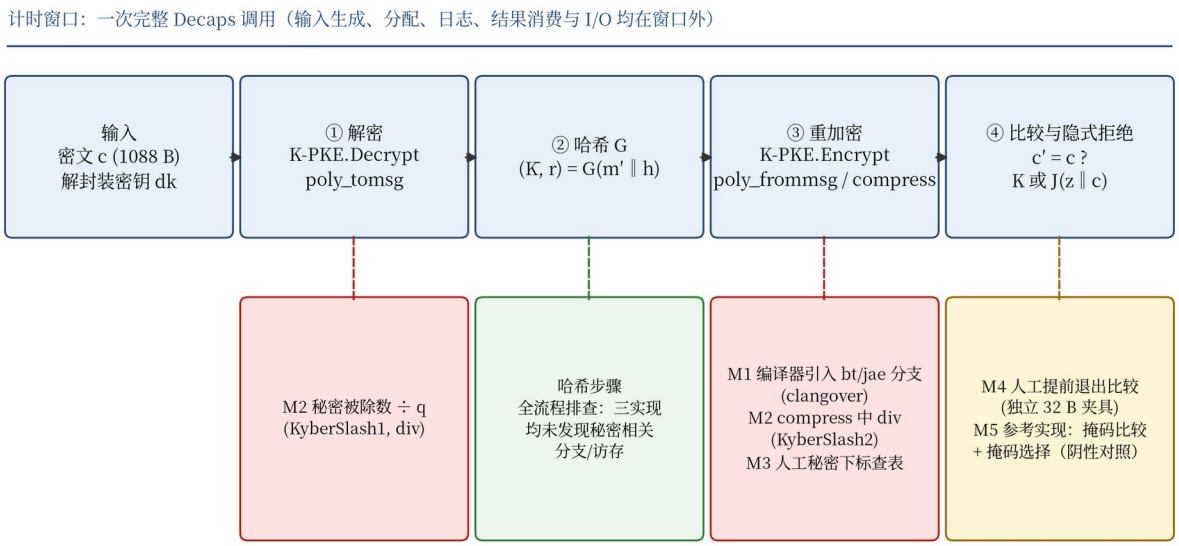


图 1 ML-KEM 解封装步骤与各模块泄漏点

按题面第 5 条，本作品把泄漏来源分为六类：①敏感条件分支；②提前返回；③可变时间比较与可变时延指令；④秘密下标访存；⑤数据相关循环；⑥异常处理路径。对 M1、M2、M5 三个实现，分别在解密、哈希 G/H/J、重加密、密文比较、隐式拒绝五个步骤上逐格排查，共 $3 \times 5 \times 6 = 90$ 格，每格给出“存在 / 不存在 / 不适用 / 未检查”之一，并附函数名、反汇编地址或源码行号（结果见 3.3.3 节）。

2.1.2 被测对象与模块设计

五个模块覆盖了题面列出的主要泄漏机制，并分为真实历史案例、人工校准和阴性对照三类（表 3）。选择历史缺陷而非自行“埋点”作为主阳性，是为了检验平台对真实编译产物的检出能力；人工夹具只用于校准检测器能否识别目标机制；阴性对照用于估计误报。

表 3 被测对象与模块设计

模块	对象与固定版本	编译条件	泄漏机制	修复 / 对照
M1	liboqs 0.10.0（ML-KEM-768-ipd 可移植 C 路径）	clang 18.1.3，-Os -fno-vectorize，显式关闭 AVX2 分发	编译器在 poly_frommsg 引入 bt/jae 条件分支 [4] [5]	官方修复 982c762
M2	pq-crystals Kyber768 参考实现，未修 a621b8d	clang 18.1.3，-Oz -g -fno-vectorize -fno-slp-vectorize	秘密被除数除以 q（KyberSlash1/2）[3]	官方修复 272125f（dda29cc 仅修第一处，不作完整修复）
M3	基于 mlkem-native，在 poly_frommsg 插入秘密下标查表	同 M5	秘密下标访存（人工）	on / off 两臂，输出逐字节相同
M4	独立 32 B 比较函数	夹具独立编译	发现不同字节即返回（人工）	vulnerable / ct 两臂
M5	mlkem-native b3ba7b3 [20]	参考 C 路径	设计为常数时间	阴性对照

M1 构建中需要特别注意：只设置可移植构建开关并不能保证运行可移植 C 版本，运行时仍可能分发到 AVX2 实现，因此必须同时关闭 AVX2 指令与 ML-KEM-768-ipd 的 AVX2 实现开关，并通过反汇编核对实际调用的 poly_frommsg。M2 的除法指令只在 -Oz 构建中出现，换优化级别重建可能使机制静默消失，因此采集程序记录并核对二进制摘要。

2.2 平台总体架构

平台的总体架构如图 2 所示，自上而下分为冻结计划、被测模块、测量层、统计 / 定位 / 族外交叉三个分析层，以及结果登记层。所有模块共用同一套测量入口、统计电池和结果格式，结论统一登记为“检出 / 未检出（附 MDE）/ 未执行 / 不适用”四种状态之一，避免把“没有做”写成“没有问题”。

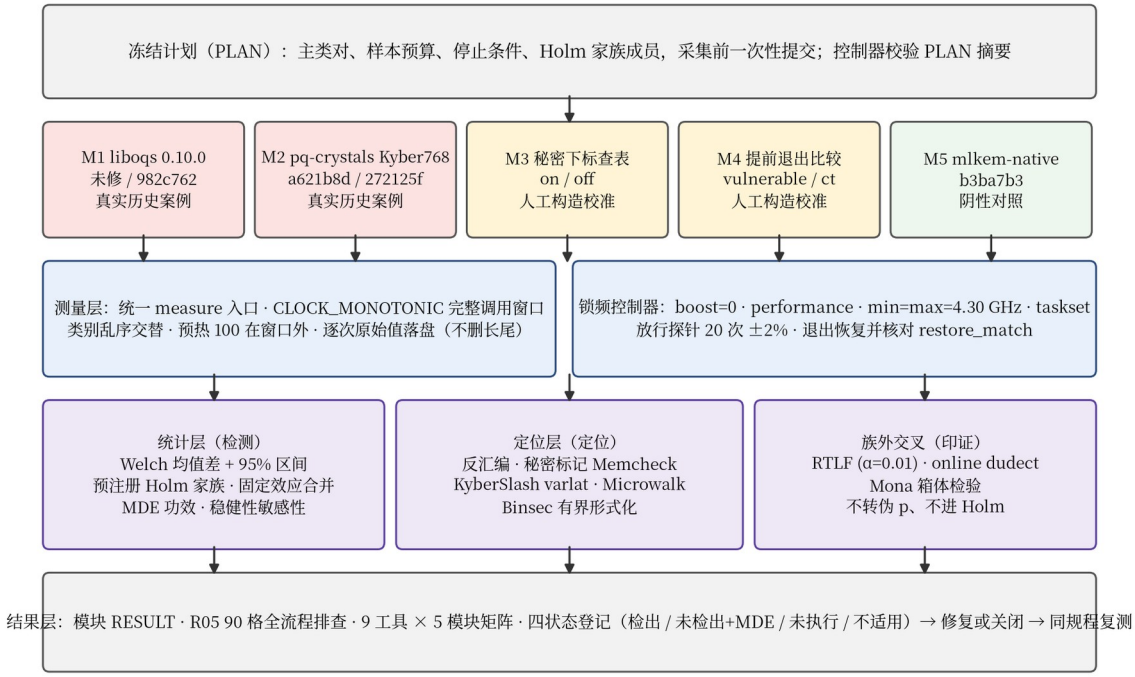


图 2 平台总体架构

2.3 测量层设计

2.3.1 计时窗口与采样规程

计时使用 CLOCK_MONOTONIC，窗口只包住一次完整解封装调用。每次运行先在窗口外空跑 100 次预热；各类输入事先按种子乱序后交替执行，避免“先测完 A 再测 B”引入的时间漂移；每类 10000 次，逐次原始值写入 CSV，并在元数据中记录种子、CPU、编译器、选项、时钟单位与二进制摘要。普通长尾一律保留，异常样本规则在计划中预先写定，稳健性另由敏感性分析检查（3.4.3 节）。

本机读钟开销约 17.4 ns，最小步进 10 ns，有效计时量化尺度约 10 ns。这一分辨率与本作品关注的效应处于同一量级，是限制检测灵敏度的因素之一已列入局限（5.2 节）。

2.3.2 锁频控制器

控制器在每次正式采集前执行：①关闭全局睿频（boost = 0）；②将调速策略设为 performance，最低、最高频率均设为 4300000 kHz；③用 taskset 把被测进程绑定到 CPU2（复测换 CPU6）；④等待 1.0 s 后以 50 ms 间隔读取

20 次当前频率，全部落在目标值 ±2% 以内且跨度不超过 50000 kHz 才放行；⑤采集结束后恢复全部设置，并核对恢复后状态与进入前一致（restore_match）。控制器保存进入、采集中、退出三个阶段的记录。

需要说明，控制器给出的是“请求频率与可见回读一致”的工程控制观测，不等于逐调用的物理恒频保证。本机未配置 isolcpus，SMT 处于开启状态，后台约 39 个服务在运行，这些条件均如实记录。

2.3.3 独立运行设计

每个模块做三次独立运行：r1（种子 31337，CPU2）、r2（种子 424242，CPU2）、r3（种子 31337，CPU6）。换种子用于检查结论是否依赖特定输入，换核用于检查是否依赖特定物理核。需要指出的局限是：主类对 message-zero 与 message-ones 各自是同一份确定性密文的 10000 次重复，因此单次运行的区间反映的是测量噪声，而不是密文间的变异；跨种子复测部分弥补了这一点，但并不等价于在每类内部使用大量不同密文。

2.4 统计层设计

2.4.1 检验统计量与区间

对每个类对，计算两类样本均值差 Δ、Welch t 统计量与自由度，95% 区间与 p 值采用大样本正态近似（z = 1.9599639845）。由于每类样本量为 10000，正态近似误差可以忽略，但本文不将其称为有限样本精确的 Student-t 区间。零方差、数据不足或工具缺失单列状态，不并入检验。

2.4.2 预注册检验家族与多重校正

每个模块的检验预先归为一个家族，在全族 α = 0.05 下做 Holm 校正：M1、M2、M3、M4 各 6 项（3 次运行 × 2 个版本或臂），M5 为 3 项。M1 沿用前期阶段封存的家族；M2–M5 的四份计划在采集前由同一提交一次冻结，控制器每次运行校验计划文件的 SHA-256 摘要，审计时再独立复算。预注册的作用是防止“看了数据再改规则”，例如不显著就追加样本，或更换分组直到显著。

2.4.3 跨运行合并与异质性

三次运行的结果按逆方差固定效应合并：以各运行区间反推标准误 $SE_i = (CI_{high} - CI_{low})/(2z)$，权重 $w_i = 1/SE_i^2$，合并估计 $\Delta_{pool} = \Sigma w_i\Delta_i / \Sigma w_i$，合并标准误为 $(\Sigma w_i)^{(-1/2)}$；同时报告 I^2 异质性。I^2 较高（如 M2 的 58.3%）提示运行间存在未受控漂移，此时合并区间只作描述，判定以预注册的 Holm 结果为准。

2.4.4 最小可检测效应

对每个“未检出”的家族，按实际样本量和观测方差给出在 α 与功效约定下可检测的最小均值差 MDE。“未检出”的确切含义是“真实效应若存在，其大小很可能低于 MDE”，而不是“效应为 0”。

2.4.5 族外交叉检验

RTLF [9]（$\alpha = 0.01$，固定种子 2026101605，bootstrap 10000 次）检验分布差异（含分位与尾部）；online dudect [6] 在线计算最大 t 统计量；Mona 做箱体分离检验。三者一律放在 Holm 家族之外，其二值结论、最大 t 或箱体结论都不转换为 p 值参与校正，只作独立交叉印证。Welch 检验比较的是均值，RTLF 比较的是分布，两者互不替代。

2.5 定位层设计

（1）反汇编检查：对实际被计时的 ELF 做 objdump，检查目标函数中的条件跳转、div/idiv 指令和秘密下标寻址，并记录指令地址与字节摘要。

（2）秘密标记 Memcheck：沿用 ctgrind [16] 的思路，把私钥中参与解密的 1152 B（M5 另加隐式拒绝秘密 z 的 32 B）标记为“未初始化”，程序一旦以秘密值作分支条件或访存下标，Memcheck 即报告具体源码行。

（3）可变时延指令检查：使用 KyberSlash 作者发布的 Valgrind 补丁（varlat，基于 Valgrind 3.25.1），报告秘密值进入除法等可变时延指令的位置，用于 M2 定位与 M5 复查。

（4）Microwalk [14]：借助 Intel Pin 3.28 记录执行轨迹，以轨迹与秘密之间的互信息对可疑指令排序，用于 M3。

（5）Binsec [15]：对 M4 抽取的比较内核做关系符号执行（i386，-O1，公开长度 1–16 B），判定控制流和访存是否只依赖公开输入，并在 insecure 时给出反例地址。为避免误读，额外检查了探索完整性：将搜索深度限为 5 时工具返回 unknown 而进程退出码仍为 0，说明不能仅凭退出码判定安全。

各工具的定位结论均不等同于常数时间证明：动态工具只覆盖实际执行过的输入，Binsec 的 secure 只在给定边界内成立。

2.6 防护设计与泄露渠道审计

2.6.1 防护手段

本作品覆盖题面第 6 条列出的七类防护手段，并核验每一项在机器码层面是否生效（表 4）。

表 4 防护手段与产物级核验

防护手段	对应模块	实现方式	产物级核验
消除数据相关分支	M1	官方修复 982c762：poly_frommsg 中先清零系数，再以 cmov_int16 按消息位做掩码选择	未修 bt/jae → 修复后 bt + setb + call cmov_int16，函数内无 jae；Memcheck 1 → 0
掩码选择（cmov）	M1、M5	cmov_int16：b = −b；*r ^= b & (*r ^ v)；隐式拒绝时对 K 与 J(z‖c) 做字节掩码选择	cmov_int16 机器码为 neg/xor/and/xor；隐式拒绝掩码由 sete 生成，解封装函数体内 0 条跳转
消除数据相关除法	M2	272125f：以乘法与移位替代 /KYBER_Q（poly_tomsg、poly_compress、polyvec_compress）	三处 div 计数 1 → 0；varlat 1280/3 → 0/0
消除数据相关访存	M3	off 臂去掉秘密下标查表	Memcheck 1280/2 → 0/0；互信息 1.922 → 0.722 bits
常数时间比较	M4	逐字节异或后按位或累积，循环结束后统一判断	Memcheck 32/1 → 0；Binsec insecure → 有界 secure
固定循环上界	M4、M5	比较循环始终遍历公开长度	反汇编与 Binsec 有界检查
统一错误处理（隐式拒绝）	M2、M5	比较失败时输出 J(z‖c)，返回路径与成功路径一致	功能层：1000 例额外输入拒绝 1000/1000；泄露渠道审计

M1 修复的核心代码如下（摘自官方补丁 982c762）：

```
/* poly_frommsg, 修复后 */
r->coeffs[8*i+j] = 0;
cmov_int16(r->coeffs+8*i+j, ((KYBER_Q+1)/2), (msg[i]>>j)&1);

void cmov_int16(int16_t *r, int16_t v, uint16_t b) {
  b = -b;
  *r ^= b & ((*r) ^ v);
}
```

M4 两臂的差异如下：

```
/* vulnerable 臂: 发现不同即返回 */
for (i = 0; i < len; i++)
  if (secret_bytes[i] != candidate_bytes[i]) return 0;
return 1;

/* ct 臂: 遍历全部字节后统一判断 */
unsigned int difference_bits = 0;
for (i = 0; i < len; i++)
  difference_bits |= (unsigned int)(secret_bytes[i] ^ candidate_bytes[i]);
return difference_bits == 0;
```

需要说明，本作品的真实缺陷防护均采用上游官方修复，平台的作用是在同一规程下核验修复在产物层面是否生效，而不是提出新的修复算法。M5 参考实现在掩码比较中使用了值屏障（mlk_value_barrier）以阻止编译器把掩码运算还原为分支；M1 所用 liboqs 版本的 cmov 未带值屏障，在本作品主构建（-Os）的解封装函数体内未观测到由此产生的分支，但它在其他编译器与选项下仍是一个未经验证的可移植性隐患（见 5.2 节）。

2.6.2 日志、异常与返回码泄露审计

为回答“防护过程本身是否通过日志、异常或返回码泄露秘密”，对 128 个源文件中的 1048 个输出或返回调用点做静态扫描，其中 122 个接触秘密相关数据；结论为秘密内容泄露 0 条。审计发现 1 条低危问题：功能自检分支会使计时程序在不同构建下退出码不同，构成可观察信号。修复方式是把自检改为仅在显式参数下执行，计时模式下标准错误恒空、退出码只反映参数、I/O 或内存分配失败。修复后“内部验证结果泄露”类由 1 条降为 0 条；动态差分对 7 种输入模式各运行 120 次，返回码、标准输出与标准错误逐字节一致。审计未覆盖上游第三方代码、网络路径以及进程级元数据（打开的文件描述符内存映射、资源用量等），这些需要单独专项检查。

2.7 运行结果

平台在本机上完成了全部五个模块的正式采集。M1 数据为前期阶段在同一机器、同一锁频规程下采集并封存的结果，本文直接引用；M2–M5 正式采集共 54 次控制器运行，另有 6 次 M2 微基准运行；收尾阶段另有 117 次控制器运行（格式破损对照 18 次、性能开销 54 次、优化级别对照 36 次、锁频对照 9 次），全部通过放行探针与恢复核对。以性能开销实验的 54 次运行为例，逐次实测中位频率的中位数为 4309722.5 kHz，相对名义 4.30 GHz 偏差 +0.005%；其中 1 次运行在采集期间的频率跨度为 50310 kHz，略超 50000 kHz 的跨度阈值，但 20 个采样值仍在 ±2% 带内，已如实登记。

2.8 技术指标

表 5 列出平台的主要技术指标，均为本机实测值。

表 5 平台主要技术指标

指标	实测值
测量对象	一次完整 ML-KEM-768 解封装（M1 未修均值 57010.7 ns，M5 均值 29455.7 ns）
计时分辨率	读钟开销约 17.4 ns，最小步进 10 ns，有效量化尺度约 10 ns
样本规模	每类 10000 次 × 3 次独立运行；窗口外预热 100 次
已检出的最小效应	M1 完整解封装 +32.3 ns（约 0.06%）；M4 比较调用 −4.09 ns
未检出项的 MDE	M2 25–67 ns；M3 18–34 ns；M5 14–28 ns；M1（Holm 口径）31.39 ns；M4 最坏 1.47 ns
阴性对照误报	M5 上运行过的 6 种工具误报 0
定位覆盖	90 格全流程排查，未检查 0 格；9 工具 × 5 模块矩阵中 30 格有运行产物
防护开销（解封装）	M1 +1.51% [1.36, 1.65]；M2 −2.92% [−3.06, −2.76]
功能正确性	M2 官方 KAT 100/100；M3、M5 各 80/80；M1 执行 60 例通过 35（keygen 0/25 为 ipd 与最终稿差异所致）
复现	干净克隆 194/194；匿名包内 CTest 5/5；quick 档 263/263（仅验证软件流程，不含计时结论）

3 系统测试与结果

3.1 测试方案

测试分为六组：①功能正确性与修复语义等价（题面第 1 条）；②检测与定位（第 3–5 条）；③防护后复测（第 6–7 条）；④功效、稳健性与异常输入对照（第 2–4 条）；⑤性能开销（第 8 条）；⑥跨编译选项、跨环境与多工具对照（第 9、12 条）。测试环境见表 6。

表 6 测试环境

项目	配置
处理器	AMD Ryzen 9 9950X（x86-64），SMT 开启，未配置 isolcpus
操作系统	Ubuntu 24.04.4 LTS，Linux 6.17，amd_pstate 频率驱动
编译器	clang 18.1.3（主构建）；gcc 13.3.0 已安装但未用于对照实验
动态分析	Valgrind 3.22.0（Memcheck）；Valgrind 3.25.1 + KyberSlash varlat 补丁；Intel Pin 3.28（Microwalk）
形式化	Binsec 0.11.3 + Z3 4.15.3
计时源	CLOCK_MONOTONIC（完整解封装）；CLOCK_MONOTONIC_RAW（M4 比较调用）
频率控制	boost = 0，performance，min = max = 4300000 kHz，taskset CPU2 / CPU6

3.2 功能测试

功能测试用于确认被测实现及其修复版本均保持 ML-KEM 的正确性与安全语义，防止“以改变算法语义换取常数时间”（题面第 1 条）。结果见表 7。

表 7 功能正确性与修复语义等价

对象	测试向量	结果与说明
M1 两版	NIST ACVP 转换的 ML-KEM-768 向量	执行 60 例通过 35：encaps 25/25、decaps 10/10（含 5 例隐式拒绝）通过；keygen 0/25 未通过，原因是 liboqs 0.10.0 实现的是 FIPS 203 初始公开草案，其 K-PKE.KeyGen 的哈希输入不含最终稿新增的参数集域分隔字节，属于草案与最终稿的真实差异；另有 20 例密钥校验因该封装层无对应入口记为不适用

对象	测试向量	结果与说明
M2 两版	Kyber round3 官方 KAT	100/100；普通构建与 ASan/UBSan 构建测试各 7/7
M3、M5	NIST 独立向量	各 80/80（含隐式拒绝用例）
修复等价性	同一驱动、同一公开输入	三对“未修 / 已修”实现的逐例输出摘要 diff 全空；另以 1000 例额外输入（一半随机密文、一半逐字节篡改密文）测试，隐式拒绝 1000/1000 按 FIPS 203 或 round3 规范生效

3.3 检测与定位结果

3.3.1 端到端检测

图 3 汇总各模块主类对在完整解封装（M4 为比较调用）上的均值差与 95% 区间。M1 未修版本三次运行的均值差分别为 +39.337、+31.117、+22.337 ns，三次区间均不含 0，合并 +32.311 ns [20.282, 44.339]，在 m = 6 的 Holm 家族中唯一被拒绝（$p = 1.30\times10^{-5}$，阈值 0.00833）；族外 dudect 最大 t 为 26.41、40.64、14.92（上游阈值 10），RTLF 同向判定有差异。M4 vulnerable 臂三次原始 p 为 0（双精度下溢）、9.23×10^{-25}、3.14×10^{-22}，合并 −4.089 ns [−4.215, −3.963]，负号表示首字节即不同的输入更快，与提前退出机制一致；dudect 最大 t 为 356.6、396.9、395.6。M2、M3、M5 的预注册家族均未拒绝任何检验。

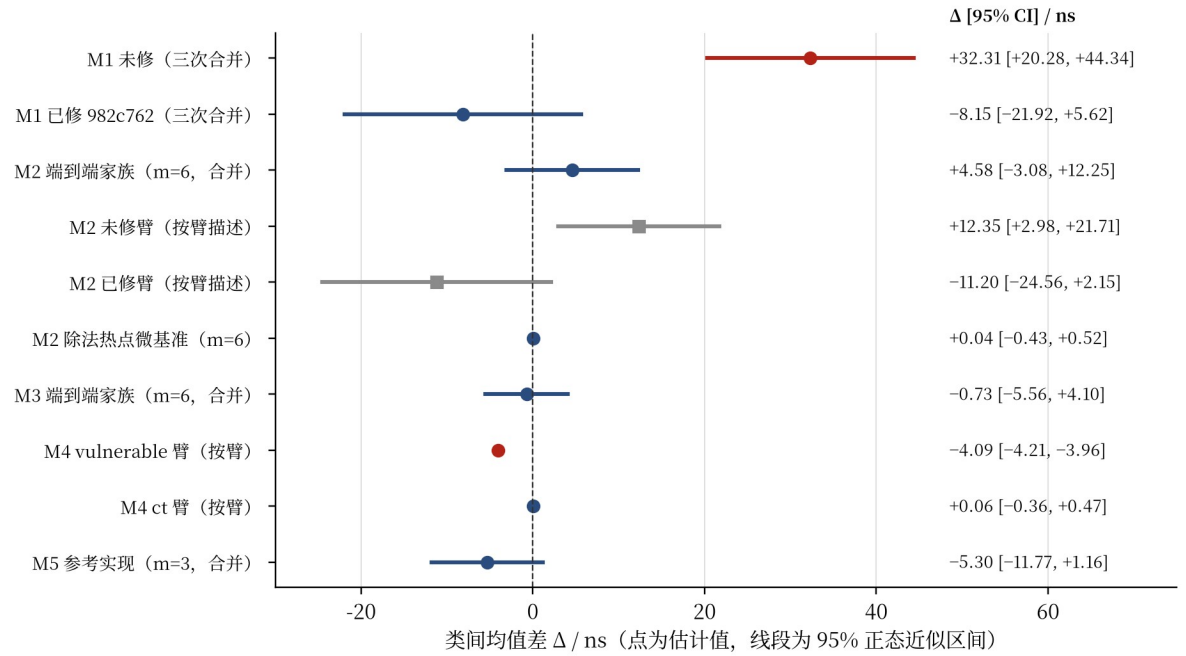


图 3 各模块主类对均值差及 95% 区间

注：红色为检出，蓝色为未检出，灰色为按臂的描述性数值（不作判定）。M1 数据为前期封存结果；M4 为比较调用计时，量纲虽同为 ns，但与完整解封装不可直接比较。

关于效应的符号需要说明：端到端 Δ 的符号受二进制布局、对齐等因素影响，同一 -Os 机制在不同构建中曾给出相反符号（主实验 +32.3 ns，优化级别实验中按 zero − ones 定义为 −26.16 ns），因此本作品只依据“是否可区分”作判定，不用分支代价模型解释符号。

3.3.2 函数级定位

（1）M1：反汇编显示，未修版本 poly_frommsg 中出现 bt %ecx,%r8d 后接 jae 的按位条件跳转，判据正是消息位；秘密标记 Memcheck 在 poly.c:179 报告 1 个秘密相关条件跳转，调用栈经由重加密函数 indcpa_enc。修复后该位置变为 bt + setb + cmov_int16，Memcheck 报告 0 个。

（2）M2：反汇编显示，未修版本 poly_tomsg、poly_compress、polyvec_compress 三处各有 1 条以秘密值为被除数、以 3329 为除数的 div 指令；varlat 报告 1280 errors / 3 contexts，分别指向 poly.c:188、poly.c:28 与 polyvec.c:48。修复后三处 div 计数均为 0，varlat 0/0。关闭 varlat 时两版均为 0，说明普通 Memcheck 无法发现可变时延除法，这正是需要专门工具的原因。

（3）M3：秘密标记 Memcheck 报告 1280 errors / 2 contexts，均落在人工插入的秘密下标读取上；Microwalk 对 poly_frommsg 两处读取给出 1.922 bits 的互信息，为全场最高，off 臂同函数为 0.722 bits。

（4）M4：Memcheck 报告 32 errors / 1 context（compare_early.c:9）；Binsec 判定 vulnerable 为 insecure，反例地址 0x8049047 恰好对应 compare_early.c:9 的提前返回，ct 臂判定为有界 secure 且所有路径探索完整。

（5）M5：将私钥秘密部分（1152 B）与隐式拒绝秘密 z（32 B）标记后，在普通 Memcheck、varlat 关、varlat 开三种模式下各运行 10 次完整解封装，均为 0 errors。

3.3.3 全流程六类来源排查

90 格排查的汇总见表 8。“存在”的 5 格分别为：① M1 未修 / 重加密 / 敏感分支（poly_frommsg 的 bt/jae，982c762 修复）；② M2 未修 / 解密 / 可变时延指令（poly_tomsg 的 div，dda29cc 与 272125f 修复）；③ M2 未修 / 重加密 / 可变时延指令（poly_compress、polyvec_compress 的 div，272125f 修复）；④ M2 未修与已修 / 重加密 / 敏感分支（poly_frommsg 的 bt/jae，272125f 未触及该函数）；⑤ M5 / 隐式拒绝 / 提前返回（mlkem-native 按 FIPS 203 对私钥内嵌哈希做的有意检查，检查失败时提前返回；该分支只反映“私钥是否通过完整性检查”，不依赖密文或解密消息，属上游设计行为，按有界例外登记）。

表 8 全流程排查汇总（3 实现 × 5 步骤 × 6 类来源）

实现	存在	不存在	不适用	未检查	小计
M1 liboqs 0.10.0（未修 / 982c762）	1	26	3	0	30
M2 Kyber768（a621b8d / 272125f）	3	24	3	0	30
M5 mlkem-native b3ba7b3	1	25	4	0	30
合计	5	75	10	0	90

注：“不适用”10 格均为“异常处理”类与不可能失败的函数的组合。“不存在”是在给定产物、给定输入和给定工具下未发现，不等于常数时间证明；M5 重加密步骤的 Binsec 检查结果为 unknown，未计为“不存在”的证据。人工夹具 M3、M4 不计入本表。

第④格是排查中最值得关注的发现：KyberSlash 官方修复提交 272125f 的产物中，poly_frommsg 的 bt/jae 分支与未修版本逐字节相同（两侧指令字节

SHA-256 一致），秘密标记 Memcheck 在两侧各报 256 errors / 1 context，调用链为 poly_frommsg ← indcpa_enc ← 解封装。该函数体与 M1 未修版本相同，即与 clangover 同源。由于主类对 message-zero 与 message-ones 恰好使该分支在 256 次迭代中走系统性不同的路径，这为 RTLF 在 M2 两臂主类对上均报告分布差异、而在格式破损对照类对上 6/6 未报告提供了一个机制层面的候选解释。但需要同时说明：①本作品未做逐指令时序归因，不能把 RTLF 观测到的差异直接归于该分支；②已修臂三个种子的类间偏移符号翻转（+21.7、−17.1、−15.5 ns），离散度约为未修臂的 1.9 倍，未受控漂移仍是并列因素；③在同一分支存在的情况下，M2 预注册家族仍为 Holm 拒绝 0，其原因（编译选项 -Oz 与 -Os 的代码布局差异、M2 噪声水平更高等）尚未分离，列为待补实验。该观察针对的是历史提交，当前上游代码是否仍有此问题不在本作品验证范围内。

3.4 防护效果复测

防护后在与防护前完全相同的规程下复测（题面第 7 条），结果见表 9。

表 9 防护前后对比

模块	证据层	防护前	防护后
M1	完整解封装（合并）	+32.311 ns [20.282, 44.339]，3/3 区间不含 0	−8.150 ns [−21.916, 5.616]，3/3 区间含 0
M1	dudect 最大 t / RTLF	26.41、40.64、14.92 / 有差异	1.15、1.12、1.31 / 无差异
M1	定位	bt/jae；Memcheck 1	setb + cmov_int16；Memcheck 0
M2	定位	三处 div；varlat 1280 / 3	div 计数 0；varlat 0/0
M2	完整解封装（按臂描述）	+12.35 ns [2.98, 21.71]	−11.20 ns [−24.56, 2.15]
M3	定位	Memcheck 1280 / 2；1.922 bits	Memcheck 0/0；0.722 bits
M4	比较调用	−4.089 ns [−4.215, −3.963]；dudect 356.6–396.9	+0.055 ns [−0.363, 0.473]；dudect 1.90–2.27

M1 和 M4 在计时层与定位层上均实现了“修复前可区分、修复后不可区分”，满足题面“泄漏明显降低”的要求。M2 的情况不同：由于修复前在计时层就未检出，只能在定位层说“机制消失”，不能说“修复后泄漏明显降低”。M2 按臂数值方向相反（未修 +12.35 ns、已修 −11.20 ns），家族 I^2 为 58.3%，应读作臂间漂移，而不是可归因于除法的泄漏或其修复效果。

3.5 功效、稳健性与异常输入对照

3.5.1 最小可检测效应

在 $\alpha = 0.05$、功效 0.80 的约定下，$MDE = (z_{1-\alpha/2} + z_{1-\beta}) \times (s_0^2/n_0 + s_1^2/n_1)^{(1/2)}$，Holm 口径取 α/m。表 10 给出各家族的 MDE。

表 10 各家族的最小可检测效应（每类 n = 10000）

家族	MDE（α = 0.05）	MDE（Holm 口径）	判定
M1 未修	25.27 ns	31.39 ns	观测 +39.34 ns（r1）超过 MDE，可检出
M1 已修	28.63 ns	35.56 ns	观测 −13.21 ns，小于 MDE，未检出
M2 端到端	19.9–54.0 ns	24.7–67.1 ns	6 项观测值均小于 MDE，未检出
M3 端到端	14.2–27.1 ns	17.7–33.7 ns	两臂观测值均小于 MDE，未检出
M4 比较调用	0.19–1.19 ns	最坏 1.47 ns	vulnerable 臂远超 MDE；ct 臂 0/3 超过
M5 端到端	11.7–24.5 ns	13.5–28.3 ns	观测值小于 MDE，未检出

注：MDE 使用本次实测标准差，属事后功效描述，不改变任何已提交的 Holm 判定。

由此，“M2、M3、M5 端到端未检出”的确切含义是：在本机、本编译产物、本类对和本样本预算下，观测到的类间均值差小于约 25–67 ns、18–34 ns、14–28 ns，这套预算无法分辨更小的效应。若需要对 KyberSlash 这类可能只有数纳秒的效应作出判断，样本量需按 MDE 与 n 的平方根反比关系相应增加（见 5.3 节）。

3.5.2 稳健性敏感性分析

主分析保留全部样本，不删离群值、不裁尾、不扣读钟开销。为检查结论是否依赖均值型检验，对 64 个类对（其中 21 个为冻结家族成员）另用 p99.9 截尾均值差、中位数差、Hodges-Lehmann 位置差三种方法做族外敏感性分析。全样本方法与冻结家族的 Δ 最大绝对偏差为 0 ns；64 个类对中有 12 个在不同方法下发生显著性翻转（家族成员 7 个、族外对照 5 个），但没有任何一个翻转改变 Holm 判定。翻转主要来自均值型与位置型检验的功效差异，以及约 10 ns 量化尺度下秩方法的失效，不构成新的泄漏证据。

3.5.3 格式破损与截断输入对照

为回应题面第 3 条对异常密文的要求，在 M1 两版与 M5 上另做两组族外对照（三个种子，每类 10000 次，18 次控制器运行全部通过核对）。对照 A 比较合法密文与长度正确但内容破损的密文（u 字段前 960 B 置 0xFF、v 字段后 128 B 置 0x00）：9 个 Δ 介于 −3.2 与 +9.8 ns 之间，9/9 区间含 0，原始 p 均不小于 0.28，与“隐式拒绝按设计走完整重加密”的预期一致。对照 B 比较合法密文与截断为 1087 B 的密文：截断输入耗时仅 23.1–23.4 ns 且与实现无关，说明它在进入 KEM 前即被封装层的公开长度检查拒绝；Δ 约为 29.3 μs（M5）与 55.1 μs（M1）。长度是公开信息，按固定路径拒绝属于正确的接口语义，不构成秘密相关泄漏，因此该对照不进入主检验。

3.6 性能测试

性能测试覆盖各模块的 keygen、encaps、decaps 完整调用以及 M4 比较调用（题面第 8 条）。decaps 每臂 6 块 × 25000 次，keygen 与 encaps 各 30000 次；为抵消漂移，baseline 与 treatment 以 ABAB 交错成块执行，按块配对后取相对开销中位数，区间为 bootstrap 10000 次。表 11 给出防护前后解封装的典型统计量，图 4 给出成对开销。

表 11 防护前后解封装运行时间（ns）

版本	均值	中位数	p90	p99	均值（周期）
M1 未修	57010.7	56916.0	57157.0	58729.0	245146
M1 已修 982c762	57873.3	57798.0	57959.0	59522.0	248855
M2 未修 a621b8d	57878.5	57809.0	58039.0	59762.0	248878
M2 已修 272125f	56188.0	55995.0	56656.0	58088.0	241608
M5 mlkem-native	29455.7	29404.0	29455.0	31077.0	126659

注：周期按名义锁定频率 4.30 GHz 换算，仅用于跨机器粗略对齐，不代表指令数。

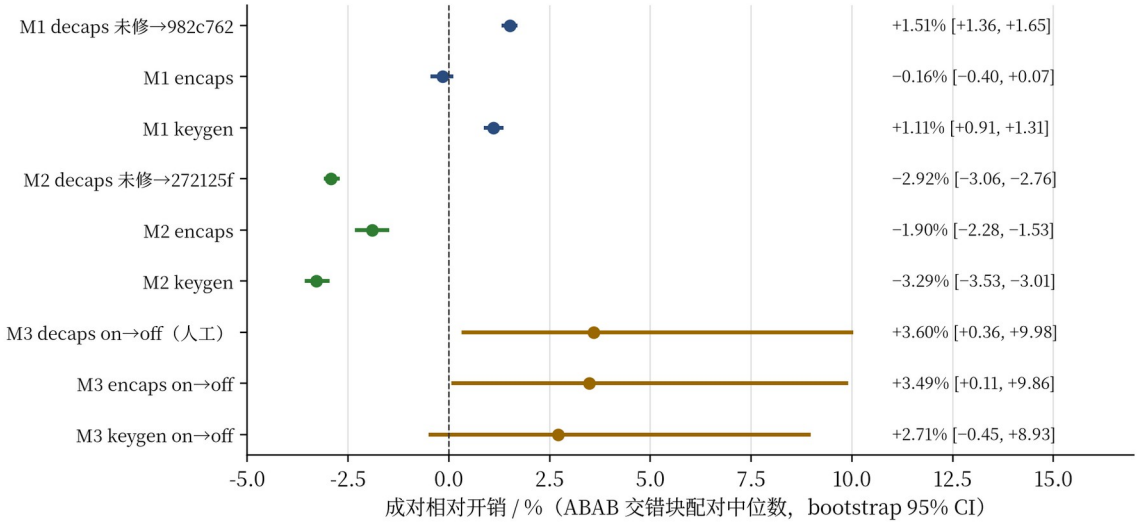


图 4 防护前后成对相对开销

M1 官方修复使解封装增加 +1.51% [1.36, 1.65]，代价主要来自 poly_frommsg 中每个系数一次 cmov_int16 函数调用；M2 修复以乘法和移位替代除法，解封装反而减少 2.92% [−3.06, −2.76]。M3 off 臂相对 on 臂的开销区间较宽（+3.60% [0.36, 9.98]），其 p99 明显偏大，提示存在长尾干扰，只作描述。M4 ct 臂相对 vulnerable 臂在首字节不同与末字节不同两种输入下分别增加 26.21% 与 6.85%，但其绝对差只有数纳秒，且只覆盖 32 B 比较调用，不外推到完整解封装。需要强调：某一版本更快不代表更安全，更慢也不代表泄漏，开销与常数时间性是两个独立问题。

3.7 跨编译选项与跨环境验证

3.7.1 编译优化级别对照

对 M1 两版与 M5 在 -O0、-O2、-O3、-Os 四个级别共 12 格，分别做产物判定（反汇编、div 计数、Memcheck）与锁频计时（每格 3 个种子，逆方差合并），结果见图 5。

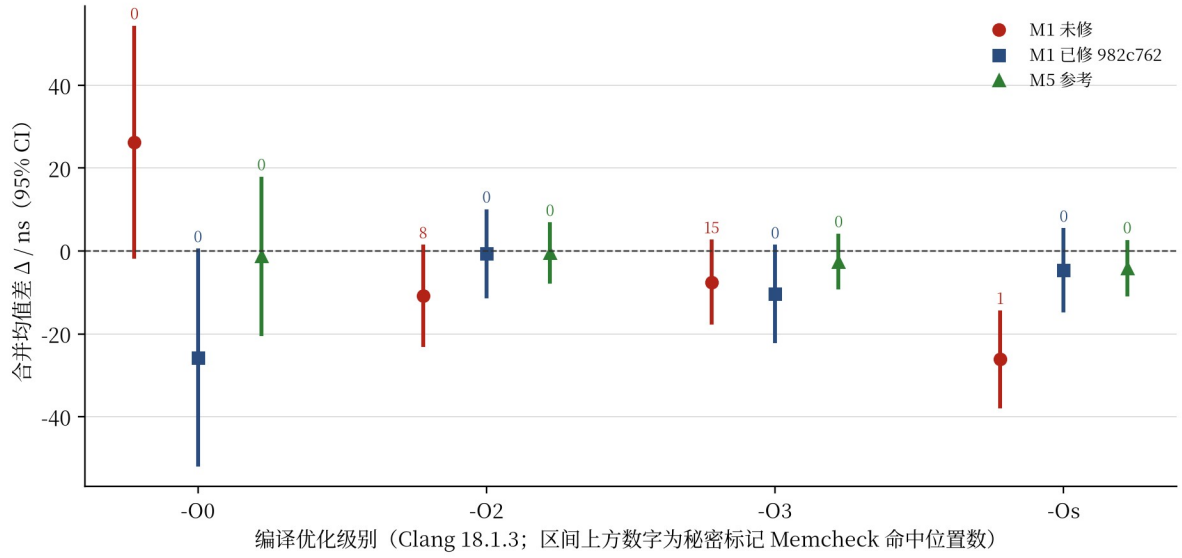


图 5 编译优化级别对照（12 格合并均值差与 95% 区间）

注：本实验的 Δ 定义为 mean(zero) − mean(ones)，与 3.3 节主实验不同，不直接比较符号。

主要结论如下：

（1）clangover 的 bt/jae 形态只在 -Os 出现，且只有 M1 未修 -Os 格的合并区间不含 0（−26.16 ns [−37.51, −14.81]），三个种子方向一致。

（2）-O2、-O3 下 M1 未修版本同样存在秘密相关分支，但形态变为每比特一条 testb 加条件跳转，Memcheck 命中位置分别为 8 和 15 个，多于 -Os 的 1 个；然而计时层合并区间均含 0。这说明“分支数更多”不意味着“计时更易检出”，定位层与计时层必须分开登记。

（3）-O0 下源码中的算术掩码被原样保留，无秘密相关分支。

（4）官方修复 982c762 在四个级别上均无秘密相关分支，计时层均未检出；M5 在四个级别上同样均无命中、均未检出。

（5）12 格中均无 div/idiv 指令。每格三个种子的二进制摘要逐字相同，并与判定所用产物的摘要对齐，保证“被计时的二进制就是被判定的那一个”。

GCC 对照以及 -O1、-Oz、-Og 级别尚未执行，列为“未执行”而非“未检出”。

3.7.2 锁频与不锁频对照

在同一机器上，对 M1 两版与 M5 各做 9 次锁频运行和 9 次不锁频运行（不锁频臂完全不写 sysfs，保持默认 boost = 1、powersave 策略），比较测量稳定性和判定一致性，结果见表 12。

表 12 锁频与不锁频对照

对象	锁频 CV	不锁频 CV	CV 比	SD 比	显著次数（锁频 / 不锁频）
M1 未修	0.011725	0.011302	0.964	0.741	3/3，3/3
M1 已修	0.013044	0.013127	1.006	0.757	1/3，0/3
M5	0.014603	0.015897	1.089	0.956	0/3，0/3

注：比值均为不锁频 / 锁频。

结果与直觉相反：不锁频时标准差反而更低（SD 比 0.741–0.956），但这主要源于不锁频时频率更高、单次调用耗时更短；换算成变异系数后两者基本相当，且方向在三个对象间不一致。M1 未修在两种条件下均为 3/3 检出，M5 均为 0/3。因此，锁频控制器的价值应表述为提供确定、可核对、可恢复的固定工作点，而不是必然降低测量方差。该对照每组仅 3 次运行，只作描述，不推断总体误报率，也不外推到其他处理器或调速策略。

3.7.3 虚拟化环境复现

为检验平台在其他软件环境中的可移植性，另在三台 Windows + WSL2 机器（不锁频、虚拟化、绑定 vCPU）上按同一执行文档复现关键实验。确定性证据完全一致：三台机器 × 12 个臂的秘密标记 Memcheck 命中上下文数与主平台相同（M1 未修 -Os 1、-O2 8、-O3 15、-O0 0；修复版与 M5 全为 0），反汇编形态逐项相同。时间证据方面，WSL2 单次运行的标准误约为数百纳秒，三台机器上 M1 未修 -Os 的合并区间均含 0；而 M4 vulnerable 臂在三台机器上的合并区间均不含 0（−7.13、−11.97、−22.37 ns），说明采集管线在效应足够大时仍能检出。该结果表明：机制存在与否可以跨环境复现，但数十纳秒级效应的可检出性强烈依赖测量环境。WSL2 数据不进入任何 Holm 家族，也不与裸金属数据合并。

3.8 多工具交叉矩阵

图 6 给出 9 种工具在 5 个模块上的运行与判定情况。45 格中 30 格有真实运行产物，15 格未运行（均登记原因，工具缺失一律写“未运行”而不写“未检出”）。

	M1 真实	M2 真实	M3 人工	M4 人工	M5 阴性
Welch+Holm（家族内）	检出	未检出	未检出	检出	未检出
RTLF α=0.01	检出	报警不可归因/方向不符	未检出	检出	未检出
online dudect	检出	报警不可归因/方向不符	未检出	检出	未检出
Mona 箱体检验	报警不可归因/方向不符	报警不可归因/方向不符	未运行	未运行	未运行
反汇编	检出	检出	检出	检出	未检出
秘密标记 Memcheck	检出	未运行	检出	检出	未检出
KyberSlash varlat	未运行	检出	未运行	未运行	未检出
Microwalk	未运行	未运行	检出	未运行	未运行
Binsec checkct	未运行	未运行	未运行	检出	未运行

图 6 多工具交叉矩阵（9 工具 × 5 模块）

交叉分析得到以下结论：

（1）M1 由反汇编、Memcheck、Welch + Holm、RTLF、dudect 五路同向印证；M4 是唯一定位层与计时层均为阳性、且对照臂在所有工具下均不报警的模块，构成“工具确实能检出”的正对照。

（2）M5 上运行过的 6 种工具全部给出阴性，误报合计 0。由于阴性对照只有一个实现、各工具的运行次数有限，该结果只说明本次未出现误报，不能据此估计总体误报率。

（3）单一工具可能导致错误结论：只看 RTLF，会把 M2 读成“两臂都可区分”；只看 dudect，会因其阳性落在已修臂（最大 t 为 36.27、30.94、61.91，未修臂为 4.68、3.05、3.66）而得出“修复引入泄漏”的结论。M2“定位阳性、计时未检出”的分层结论只有通过交叉才能稳定成立。

（4）Mona 是唯一与既有方向不一致的工具：M1、M2 未修臂均无通过验证的箱体分离，已修臂反而出现少量分离，且都出现在只剩一个验证子集的末轮，属该自适应实现的已知假阳性形态。其“置信”字段是批次成功百分比而非 p 值，本作品未用它推翻或支持任何结论。

3.9 可复现性

平台提供一键复现入口，分为两档：quick 档不需要管理员权限、不写频率节点、不绑核，完成配置、构建、测试和小样本统计演练；full 档默认只打印命令，显式指定执行参数并提供锁频控制器后才实际采集。实测结果为：干净克隆后三组 CTest 共 194 项全部通过；匿名包解压后包内 CTest 5/5 通过；quick 档在另一台 ARM 机器上 263/263 通过（只验证软件流程，不产生计时结论）。冻结计划、二进制摘要、控制器记录、原始 CSV 与统计脚本均随附，可由原始数据重算全部表格与图形。

4 与现有工作对比及应用前景

4.1 与现有工作的对比

软件时间侧信道检测大致有三条技术路线（表 13）。第一类是统计检测，以 TVLA [7][8] 和 dudect [6] 为代表，把被测程序当作黑盒，对两类输入的时间分布做 t 检验；RTLF [9] 在此基础上用 bootstrap 构造决策规则，把第一类错误率限定在预设 α 内，且与噪声分布无关。这类方法能回答“是否可区分”，但不能定位原因，并且对类构造和漂移敏感。第二类是动态分析，以 ctgrind [16] 和 Microwalk [14] 为代表，通过秘密标记或执行轨迹找出依赖秘密的分支与访存；KyberSlash 作者发布的 Valgrind 补丁进一步覆盖了可变时延除法 [3]。这类方法能定位，但只覆盖实际执行的输入，也不回答“该机制在本机上能否测到”。第三类是形式化验证，如 ct-verif [13] 和 Binsec/Rel [15]，能在给定模型和边界内证明控制流与访存只依赖公开输入，但成本高，且通常不建模指令时延。

表 13 与现有工作的对比

方法	本机可测性	函数级定位	产物级证明	未检出的量化	多机制 / 防护复测
TVLA [7][8]、dudect [6]	是	否	否	一般不报告	需另行组织
RTLF [9]	是（第一类错误有界）	否	否	随样本量提高功效	需另行组织
ctgrind [16]、Microwalk [14]	否	是	否	不适用	单一机制类
KyberSlash varlat [3]	否	是（除法）	否	不适用	针对除法
ct-verif [13]、Binsec/Rel [15]	否	是	是（有界）	不适用	单一函数或内核
本作品	是（预注册 + Holm）	是（多工具交叉）	部分（M4 有界）	报告 MDE	5 模块统一规程

本作品并不提出新的检测原语，而是把上述三类方法组织到同一个以产物为中心、可审计的流程中：对同一个被计时的二进制，同时给出统计判定、定位结果和（可行时的）有界证明，并按证据层分开登记。与 KyberSlash [3] 和 clangover [4] 等以单一机制为对象、以密钥恢复为目标的研究相比，本作品的

侧重点在于检测的可审计性和多机制覆盖，不进行密钥恢复。与 Azouaoui 等 [17] 对 Kyber 解密与重加密泄漏的系统性研究相比，后者针对功耗等物理信道，本作品针对软件计时信道，可视为互补。在统计方法上，本作品遵循泄漏评估方法学中“未检出不等于无泄漏”的原则 [8]，以 MDE 量化未检出的含义，并以 RTLF [9] 作为第一类错误有界的族外交叉检验。

本作品得到的几项经验观察对后续工作具有参考价值：① KyberSlash 的除法机制在本机上可被定位但未被计时检出，说明“定位阳性”不能直接推出“本机可利用”，也不能因计时阴性而否认机制存在；②锁频对降低变异系数的作用不明显，但对确定性和可复算是必要的；③同一 clangover 机制在不同优化级别下呈现不同指令形态，Memcheck 命中更多的级别计时反而未检出；④ dudect、RTLF、Mona 在同一数据上可能给出相互矛盾的结论，单工具结论需要交叉核验。

4.2 应用前景

（1）后量子密码库的发布前检查。平台的定位层（反汇编规则、秘密标记 Memcheck、varlat）不依赖锁频，运行成本低，适合作为密码库发布前的检查步骤；计时层可在固定机器上周期性运行，用于确认关键路径的可区分性。

（2）编译器与构建配置变更的回归检测。clangover 表明，编译器版本或优化选项的改变可能静默引入分支。平台按“二进制摘要 + 优化级别”组织判定，可用于在编译器升级或构建配置变更时比较新旧产物。将其接入持续集成（题面高水平项）尚未实现，是下一步工作。

（3）防护方案的工程评估。平台同时给出防护前后的定位结果、计时判定和成对性能开销，可为选择防护方案提供定量依据，例如 M1 修复的解封装开销约 +1.5%，M2 修复反而减少约 2.9%。

（4）教学与评测。M3、M4 校准夹具和 M5 阴性对照可作为侧信道检测工具的标准测试集，用于检验新工具能否识别典型机制、是否会误报。

5 结论

5.1 主要结论

本作品针对 ML-KEM 解封装实现，构建了检测、定位、防护、复测四段闭环的时间侧信道评估平台，并在两个真实历史缺陷、两个人工校准夹具和一个阴性对照上完成了实验。主要结论如下：

（1）平台能够在本机完整解封装上检出约 32 ns（约 0.06%）的真实泄漏。M1 未修版本合并均值差 +32.311 ns [20.282, 44.339]，在预注册 Holm 家族中被拒绝，dudect 与 RTLF 同向；官方修复后三次区间均含 0，定位层分支消失，解封装开销 +1.51%。

（2）定位层与计时层必须分开登记。KyberSlash 的三处秘密除法在定位层被稳定识别，修复后消失，但在本机完整解封装与热点微基准两层均未检出（MDE 25–67 ns）；clangover 形态在 -O2/-O3 下 Memcheck 命中更多，计时却未检出。

（3）修复已知泄漏不等于产物常数时间。90 格排查发现 KyberSlash 官方修复版本的 poly_frommsg 仍保留与 clangover 同源的秘密相关分支；这是历史提交上的机制层观察，其对计时的影响尚未归因。

（4）阴性对照未出现误报。M5 上运行过的 6 种工具误报均为 0，预注册家族未拒绝；但这只说明“在本预算下未检出”，不是常数时间证明。

（5）测量条件决定可检出性。锁频的价值在于确定性和可复算，而非显著降噪；在虚拟化环境中，确定性证据可完全复现，而数十纳秒级的计时效应不可检出。

5.2 局限性

本作品的结论受到以下限制，评估者在引用时应一并考虑：

（1）平台范围：所有计时结论仅限于一台 x86-64 机器（AMD Ryzen 9 9950X）、clang 18.1.3 和所列编译选项；未做 ARM 等其他架构，也未做 GCC 对照及 -O1/-Oz/-Og 级别的优化级别对照。

（2）网络场景：未做远程网络计时实验，无法回答题面第 10 条中远程攻击的能力边界，只能明确不作此类推广。

（3）防护的原创性：真实缺陷的防护均采用上游官方修复，平台主要承担核验作用；对 272125f 中残留的 poly_frommsg 分支，尚未实施并复测相应修复。

（4）输入设计与功效：主类对每类是同一份确定性密文的重复，区间反映的是测量噪声而非密文间变异；M2 的类对并非针对除法操作数设计，且 MDE 25–67 ns 远大于除法可能带来的效应。

（5）计时精度与隔离：有效计时量化尺度约 10 ns，未使用周期计数器；未配置 isolcpus，SMT 开启，后台服务仍在运行。

（6）工具覆盖：M5 重加密步骤的 Binsec 检查结果为 unknown；Microwalk 仅在 M3 上运行；Binsec 只对 M4 抽取内核运行，未开启乘除法检查；M1 所用 cmov 缺少值屏障的可移植性风险未经验证。

5.3 后续工作

按优先级，后续工作包括：①将 982c762 式的掩码选择移植到 272125f 的 poly_frommsg，并在同一规程下复测，形成自主防护的完整闭环；②针对除法操作数设计 M2 输入类，在每类内部使用不同密文，并按 MDE 估算扩大样本量；③引入周期计数器计时并配置 isolcpus、关闭 SMT 后复测；④补充 GCC 与其他优化级别对照，以及至少一种 ARM 平台；⑤在局域网环境做远程计时实验，给出本地与远程的能力边界；⑥把定位层检查接入持续集成，形成时间侧信道回归测试。

参考文献

[1] National Institute of Standards and Technology. FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard[S]. 2024.

[2] Bos J, Ducas L, Kiltz E, et al. CRYSTALS-Kyber: A CCA-Secure Module-Lattice-Based KEM[C]// IEEE European Symposium on Security and Privacy (EuroS&P). 2018.

[3] Bernstein D J, Bhargavan K, Bhasin S, et al. KyberSlash: Exploiting Secret-Dependent Division Timings in Kyber Implementations[J]. IACR Transactions on Cryptographic Hardware and Embedded Systems, 2025(2). IACR ePrint 2024/1049.

[4] Purnal A. clangover: Compiler-Introduced Timing Leak in the Kyber Reference Implementation[EB/OL]. https://github.com/antoonpurnal/clangover, 2024.

[5] Open Quantum Safe. liboqs Security Advisory GHSA-f2v9-5498-2vpp[EB/OL]. https://github.com/open-quantum-safe/liboqs/security/advisories/GHSA-f2v9-5498-2vpp, 2024.

[6] Reparaz O, Balasch J, Verbauwhede I. Dude, Is My Code Constant Time? [C]// Design, Automation & Test in Europe (DATE). 2017.

[7] Goodwill G, Jun B, Jaffe J, et al. A Testing Methodology for Side-Channel Resistance Validation[C]// NIST Non-Invasive Attack Testing Workshop. 2011.

[8] Schneider T, Moradi A. Leakage Assessment Methodology: A Clear Roadmap for Side-Channel Evaluations[C]// Cryptographic Hardware and Embedded Systems (CHES). 2015.

[9] Dunsche M, Maehren M, Erinola N, et al. With Great Power Come Great Side Channels: Statistical Timing Side-Channel Analyses with Bounded Type-1 Errors[C]// 33rd USENIX Security Symposium. 2024: 6687-6704.

[10] Guo Q, Johansson T, Nilsson A. A Key-Recovery Timing Attack on Post-Quantum Primitives Using the Fujisaki-Okamoto Transformation and Its Application on FrodoKEM[C]// Public-Key Cryptography (PKC). 2020.

[11] Yu T, Cheng C, Yang Z, et al. Hints from Hertz: Dynamic Frequency Scaling Side-Channel Analysis of Number Theoretic Transform in Lattice-Based KEMs[J]. IACR Transactions on Cryptographic Hardware and Embedded Systems, 2024(3).

[12] Schröder R L, Gast S, Guo Q. Divide and Surrender: Exploiting Variable Division Instruction Timing in HQC Key Recovery Attacks[C]// 33rd USENIX Security Symposium. 2024.

[13] Almeida J B, Barbosa M, Barthe G, et al. Verifying Constant-Time Implementations[C]// 25th USENIX Security Symposium. 2016.

[14] Wichelmann J, Moghimi A, Eisenbarth T, et al. MicroWalk: A Framework for Finding Side Channels in Binaries[C]// Annual Computer Security Applications Conference (ACSAC). 2018.

[15] Daniel L A, Bardin S, Rezk T. Binsec/Rel: Efficient Relational Symbolic Execution for Constant-Time at Binary-Level[C]// IEEE Symposium on Security and Privacy. 2020.

[16] Langley A. ctgrind: Checking That Functions Are Constant Time with Valgrind[EB/OL]. https://github.com/agl/ctgrind, 2010.

[17] Azouaoui M, Bronchain O, Hoffmann C, et al. Systematic Study of Decryption and Re-encryption Leakage: The Case of Kyber[C]// Selected Areas in Cryptography (SAC). 2021.

[18] Jancar J, Fourné M, De Almeida Braga D, et al. "They're Not That Hard to Mitigate": What Cryptographic Library Developers Think About Timing Attacks[C]// IEEE Symposium on Security and Privacy. 2022.

[19] Open Quantum Safe. liboqs[EB/OL]. https://github.com/open-quantum-safe/liboqs.

[20] PQ Code Package. mlkem-native[EB/OL]. https://github.com/pq-code-package/mlkem-native.

附录 A 题面要求对照

表 A1 题面 12 条要求与本报告对应章节

序号	要求	本作品的做法	章节	状态
1	基本密码功能	接入三套实现，官方 / NIST 向量验证；修复语义等价与隐式拒绝检查	3.2	完成（M1 keygen 为 ipd 差异）
2	时间测量环境	锁频控制器、绑核、窗口外预热、原始值全量落盘、环境 28 项记录	2.3、3.1	完成（未隔离核、SMT 开启）
3	输入分类与测试	主类对只差解密消息位；篡改、随机、破损、截断等族外对照	1.3、3.5.3	完成
4	侧信道统计检测	Welch、95% 区间、Holm、固定效应合并、MDE、分位数与稳健性分析、RTLF/dudect	2.4、3.3、3.5	完成
5	泄露定位	五类定位工具；90 格全流程排查	2.5、3.3.2–3.3.3	完成
6	常数时间防护	七类防护手段产物级核验；日志 / 异常 / 返回码审计	2.6	完成（真实缺陷采用上游修复）
7	防护效果验证	同规程复测；功能等价	3.4	M1、M4 完成；M2 仅定位层
8	性能要求	均值、分位数、成对开销	3.6	完成
9	跨环境验证	4 个优化级别 × 3 实现；锁频对照；三台 WSL2 复现	3.7	部分（无其他架构、无网络）
10	威胁模型	本地白盒模型与边界	1.2	完成（远程场景未实验）
11	可复现性	一键入口、冻结计划、摘要、原始数据	3.9	完成
12	创新要求	分层证据模型、预注册锁频协议、多工具交叉、优化级别对照	1.4	完成
高水平项	自动生成样本、函数级追踪、二进制检查、CI 回归	确定性种子生成样本；反汇编与 Binsec 二进制检查；Memcheck 函数级定位	2.3、2.5	CI 回归未实现

附录 B 证据索引

本报告中的全部数值均可在随附支撑材料中找到原始出处。表 B1 给出主要结论与证据文件的对应关系（路径相对于支撑材料根目录，编号与支撑材料卷一致）。

表 B1 主要结论的证据出处

结论	证据文件
M1 封存结论与 Holm 判定	[E02] docs/evidence/r02-r04-close-20261016/RESULT.md；[E17] docs/evidence/liboqs010-locked-20261015/README.md
M1 工具交叉（Memcheck、反汇编、RTLF、dudect）	[E18] docs/evidence/tooling-strengthen-20261015/RESULT.md
M2 定位与版本审计	[E03] docs/evidence/m2-kyberslash-20261016/RESULT.md；[E04] 同目录 VERSION_AUDIT.md
M2 热点微基准	[E27] docs/evidence/m2-kem-20261016/microbench-0551254c9579/RESULT.md
M3 定位（Memcheck、Microwalk）	[E21] docs/evidence/m3-completion-20261016/LOCATE_RESULT.md
M4 Binsec 有界验证	[E22] docs/evidence/m4-completion-20261016/FORMAL_RESULT.md
M5 秘密标记与 varlat	[E15] docs/evidence/m5-completion-20261016/RESULT.md
M2–M5 锁频采集与 Holm 家族	[E19] docs/evidence/platform-collection-20261016/README.md；[E20] 同目录 stats/holm/RESULT.md
RTLF 42 项	[E26] docs/evidence/platform-rtlf-20261016/RTLF_RESULT.md
Mona	[E09] docs/evidence/mona-wire-20261016/RESULT.md
功能向量与修复等价	[E32] docs/evidence/r01-kat-20261016/VECTORS.md；[E33] 同目录 EQUIV.md
环境与稳健性	[E34] docs/evidence/r02-env-20261016/ENV_TABLE.md；[E36] 同目录 ROBUSTNESS.md
格式破损与截断对照	[E28] docs/evidence/r03-malformed-20261016/RESULT.md
MDE	[E38] docs/evidence/r04-stats-20261016/POWER.md
90 格全流程排查	[E30] docs/R05-全流程排查表.md；[E41] docs/evidence/r05-locate-20261016/RESULT.md
272125f 残留分支的边界核对	[E65] docs/evidence/m2-narrative-20261020/RESULT.md
泄露渠道审计与修复映射	[E42] docs/evidence/r06-noleak-20261016/AUDIT.md；[E43] 同目录 MAPPING.md；[E63] docs/evidence/r06-noleak-fix-20261020/AUDIT.md
性能开销	[E44] docs/evidence/r08-perf-20261016/PERF.md；[E46] 同目录 perf.csv.comparison.csv

结论	证据文件
优化级别对照	[E47] docs/evidence/r09-optlevel-20261016/RESULT.md；[E48] 同目录 BUILD.md
锁频对照与多工具矩阵	[E52] docs/evidence/r12-innovation-20261016/RESULT_LOCKED_VS_UNLOCKED.md；[E54] 同目录 TOOL_MATRIX.csv
WSL2 复现	[E61] docs/evidence/wsl-repro-20261020/CROSSCHECK.md
复现入口与参数	[E55] docs/REPRODUCE-PARAMS.md；[E56] scripts/reproduce_all.sh；[E57] docs/evidence/r12-innovation-20261016/REPRODUCE-QUICK-LOG.md